



Draw It or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

1.CS 230 Project Software Design Template.....	1
Table of Contents.....	2
Document Revision History.....	2
Executive Summary.....	3
Requirements.....	3
Design Constraints.....	3
System Architecture View.....	3
Domain Model.....	3
Evaluation.....	4
Recommendations.....	5
2.Operating Platform: <Recommend an appropriate operating platform that will allow The Gaming Room to expand Draw It or Lose It to other computing environments.>.....	5
3.Operating Systems Architectures: <Describe the details of the chosen operating platform architectures.>.....	5
4.Storage Management: <Identify an appropriate storage management system to be used with the recommended operating platform.>.....	5
5.Memory Management: <Explain how the recommended operating platform uses memory management techniques for the Draw It or Lose It software.>.....	5
6.Distributed Systems and Networks: <Knowing that the client would like Draw It or Lose It to communicate between various platforms, explain how this may be accomplished with distributed software and the network that connects the devices. Consider the dependencies between the components within the distributed systems and networks (connectivity, outages, and so on).>.....	5
7.Security: <Security is a must-have for the client. Explain how to protect user information on and between various platforms. Consider the user protection and security capabilities of the recommended operating platform.>.....	5

Document Revision History

Version	Date	Author	Comments
1.0	03/19/26	Brandon Witbrodt	Initial design document for web based Draw it or Lose it application

Executive Summary

The Gaming Room would like to expand its existing Android game, Draw It or Lose It, into a web-based application that can support multiple platforms. The goal of this project is to design a software solution that allows users on different operating systems and devices to access the same game while maintaining reliable performance, security, and scalability. The client's game involves teams competing to solve puzzles based on gradually rendered images, so the new system must support multiple games, multiple teams per game, and multiple players per team. The design also requires careful memory management, with only one instance of the game service existing in memory at a time. To support this, the design uses object-oriented principles and patterns such as Singleton and Iterator to manage shared services efficiently and maintain organized game data

Requirements

The client has both business and technical requirements that must be satisfied by the software. From a business perspective, the application must allow one or more teams to participate in a game, and each team must allow multiple players. The game should preserve the core Draw It or Lose It experience while making it available to more users on more platforms. The client also wants users to be able to determine whether a game name or team name is already in use before creating one. From a technical perspective, the software must support a web-based distributed environment so that multiple users on different systems can access the application. Only one instance of the game service should exist in memory at a time, which can be addressed through the singleton design pattern. The application must also assign unique identifiers to game, team, and player objects. The design should be maintainable, flexible, and secure enough to support future growth as the client expands beyond its current Android-only version.

Design Constraints

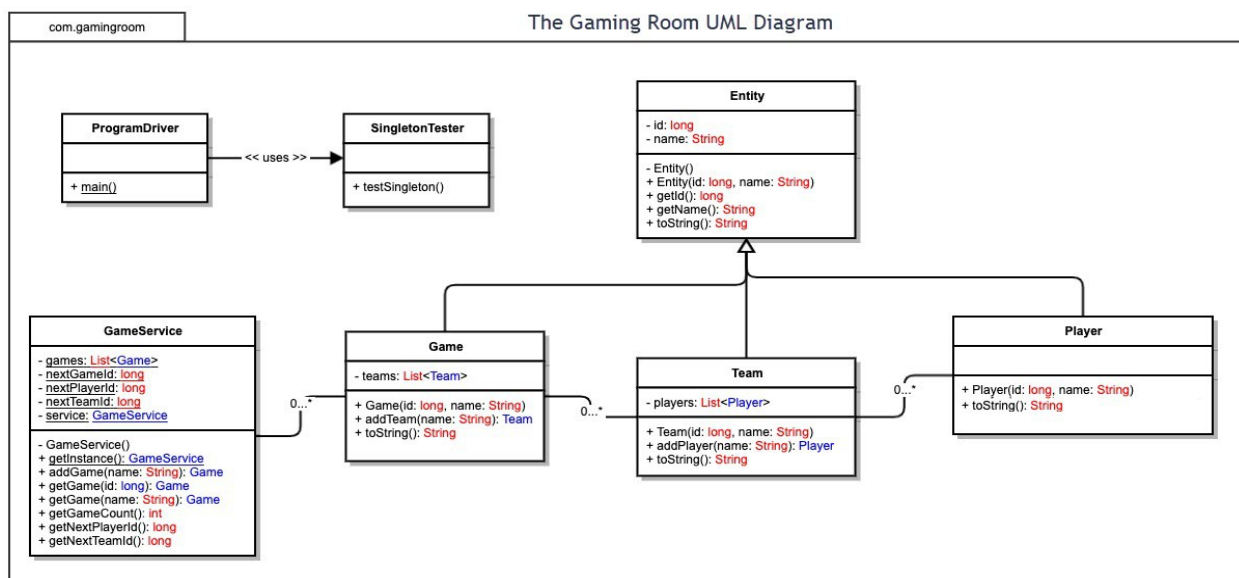
One major design constraint is that Draw It or Lose It must be developed as a web-based application instead of remaining limited to Android. This means the system must support multiple users across different devices and operating systems while working in a distributed environment. Because of this, the design must account for network communication, browser access, and the ability to scale as more users join the system. Another important constraint is the way the application manages data and memory. Only one instance of the game service should exist in memory at a time, which requires the use of the Singleton design pattern. In addition, game and team names must be unique, so the system needs logic to check existing names before creating new objects. These requirements directly affect how the classes are organized and how the application handles games, teams, and players.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The domain model for Draw It or Lose It shows the main classes and relationships needed to support the game structure. At the top of the model is the Entity class, which provides common attributes such as an identifier and a name. The Game, Team, and Player classes inherit from Entity, allowing them to share these common properties while still representing different parts of the system. This use of inheritance helps reduce repeated code and keeps the design organized. The model also shows how the classes relate to one another. A Game can contain one or more Team objects, and each Team can contain multiple Player objects. These relationships reflect the rules of the game, where teams participate in a game and players belong to teams. Overall, the UML diagram presents a simple object-oriented structure that uses inheritance and association to represent the main parts of the application clearly and efficiently.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can host a web-based application, but it is not ideal for large-scale production. It is stable and supports common web server tools, but Apple hardware is costly and less common in enterprise hosting. Mac works better for development and testing than for scalable server deployment.	Linux is the best choice for hosting a web-based application. It is reliable, scalable, secure, and widely used for web servers. It also has low licensing costs, but it may require more technical skill to manage and maintain.	Windows can host a web-based application and is supported by enterprise server tools. It is familiar to many developers and works well with Microsoft technologies, but licensing costs are higher and it usually requires more system resources than Linux.	Mobile devices are not well suited for hosting a web-based application. They are designed for client use, not server deployment, and have limits in performance, storage, reliability, and scalability for supporting many players.

Development Requirements	Mac	Linux	Windows	Mobile
Client Side	Supporting Mac clients requires testing the web application in Mac browsers such as Safari and Chrome to ensure the interface is responsive and works correctly. This adds time and testing effort, and developers may need experience with browser compatibility issues, but costs stay moderate because the same web-based system can be shared across platforms.	Supporting Linux clients requires testing the web application across common Linux browsers to confirm compatibility and responsive design. Costs remain fairly low because the same web-based client can be reused, but extra time and technical skill are needed to handle browser differences and user environments.	Supporting Windows clients requires testing the web application in common Windows browsers such as Chrome, Edge, and Firefox. Costs stay moderate because the same web-based interface can be used, but added time and developer expertise are needed to ensure consistent performance across browsers and system settings.	Supporting mobile devices requires responsive design, touch-friendly controls, and testing across Android and iOS browsers and screen sizes. This increases development time and testing effort, and it may require more expertise, but costs are reduced by using one shared web-based client instead of separate native apps.
Development Tools	Mac development can use languages such as Java, JavaScript, HTML, and CSS with tools like IntelliJ IDEA, VS Code, Eclipse, and Git. These tools support web application development well, though some Apple-specific testing may require Mac hardware, which can increase cost.	Linux development can use languages such as Java, JavaScript, HTML, and CSS with tools like IntelliJ IDEA, VS Code, Eclipse, Git, and Apache or Nginx. Most of these tools are low cost or open source, but the team needs Linux knowledge to manage and deploy them effectively.	Windows development can use languages such as Java, JavaScript, HTML, and CSS with tools like Visual Studio, IntelliJ IDEA, VS Code, Eclipse, and Git. These tools are widely supported and familiar, but some Microsoft tools and licenses can increase development cost.	Mobile development can use HTML, CSS, JavaScript, and Java with tools like VS Code, Android Studio, Xcode, and browser testing tools. These support responsive web applications well, but testing across many devices and platforms increases time, expertise, and possible licensing costs.

Recommendations

1. **Operating Platform:** Linux is the most appropriate server platform for *Draw It or Lose It*. Although the game will run on iPhone, Android, web, and PC, all versions rely on a shared backend to manage game data, user accounts, sessions, and communication. Linux is well suited for this role due to its reliability, scalability, security, and low cost. It also integrates well with modern development frameworks, databases, and cloud infrastructure, making it ideal for a distributed, multi-platform system.
2. **Operating Systems Architectures:** Linux uses a multi-user, multitasking architecture that allows multiple services, users, and game sessions to run concurrently without conflict. Its layered design consisting of the kernel, system libraries, utilities, and application layer provides efficient control over hardware resources, memory, and networking. Features like process isolation and permission control improve system stability and security, making Linux highly effective for client-server environments.
3. **Storage Management:** A relational database management system such as MySQL or PostgreSQL is recommended for storing structured data like users, teams, game sessions, and scores. This approach ensures data consistency, enforces relationships between entities, and supports multi-user access across platforms. These databases are highly compatible with Linux and offer strong performance, scalability, and security for managing persistent game data.
4. **Memory Management:** Linux uses advanced memory management techniques, including virtual memory, paging, and process isolation. These features allow multiple server processes and game sessions to run efficiently without exhausting system resources. Memory is dynamically allocated based on demand, and inactive data can be moved to disk if needed. This ensures stability and scalability as more players connect to the system.
5. **Distributed Systems and Networks:** The system should follow a client-server architecture, where each platform acts as a client connecting to a centralized Linux server. Communication should use web-based APIs (REST over HTTP/HTTPS) or real-time protocols like WebSockets to maintain synchronized gameplay. Reliable networking, error handling (such as reconnect logic), and scalability through cloud infrastructure are essential for maintaining consistent performance across platforms.
6. **Security:** Security is critical in a multi-platform environment. Linux provides strong built-in protections through user permissions, process isolation, and security frameworks like SELinux or AppArmor. Data should be encrypted using HTTPS/TLS, and secure authentication methods (such as hashed passwords and token-based sessions) should be implemented. Additional protections, including firewalls, monitoring, and regular backups, help safeguard against unauthorized access, data breaches, and service disruptions.